



Assignment - Python_Dja...
Document

research to understand the assignment in depth like what to do, what not to do, and why etc

I reviewed the assignment itself and cross-checked the testing expectations against current Django/DRF documentation. The important point is that this is **not primarily a CRUD Django assignment**. It is a small backend engineering problem where the evaluator is likely looking at your **reasoning, data modeling, box-selection algorithm, edge cases, tests, and evidence that you actually did the work**.

1. What the assignment is actually asking

The core problem is:

Given an order containing products with dimensions and weight, determine which available shipping box is most suitable.

The assignment establishes four fundamental pieces of data:

- **Product**
 - dimensions
 - weight
- **Box**
 - internal dimensions
 - maximum weight
 - cost
- **Order**
 - contains one or more products
- **Recommendation**
 - determine an appropriate box for that order

The assignment explicitly describes this ecommerce/warehouse scenario and says the goal is to build a Django-based system that recommends the most suitable box.

Assignment - Python_Django Deve...

The important phrase is:

"most suitable box"

The assignment does **not** explicitly define what "most suitable" mathematically means.

That is actually one of the places where **your engineering judgment matters**.

You need to define a reasonable selection strategy and document it.

2. What is explicitly required

The assignment explicitly requires these submission items:

Requirement	Required?
Django system	☒
Box recommendation functionality	☒
GitHub repository	☒
README.md	☒
AI_USAGE.md	☒
Exported chat transcript	☒
LEARNINGS.md / learning response	☒
Test cases	☒
Test execution output	☒
GitHub Actions or TEST_OUTPUT.md	☒

The assignment specifically lists the GitHub repository, README, AI usage document, chat transcript, learning response, test cases, and test output.

Assignment - Python_Django Deve...

So this is **not finished** when the Django application works.

The repository/documentation/evidence is part of the assignment.

3. The biggest requirement: show your own thinking

This sentence is extremely important:

"Your submission must clearly show your own thinking."

Assignment - Python_Django Deve...

This changes how you should approach the project.

They are explicitly allowing:

- ChatGPT
- Claude
- Cursor
- GitHub Copilot
- similar AI tools

But they don't want:

AI generates everything → candidate submits it without understanding it.

They want evidence of:

your reasoning → implementation → testing → verification

So if you use AI for implementation, you should be able to explain:

- Why you chose your models.
- Why you chose your algorithm.
- Why you chose a particular box.
- Why you rejected other boxes.
- Why a particular edge case behaves the way it does.
- Why you used a particular Django structure.
- What AI suggested.
- What you changed.
- What AI got wrong.
- How you verified it.

4. AI usage has unusually strict requirements

The assignment explicitly asks for `AI_USAGE.md` containing:

1. AI tools used

2. Prompts you gave
3. Output you accepted
4. Output you rejected/modified
5. Mistakes AI made
6. How you verified the final code

Assignment - Python_Django Deve...

This is different from a normal project.

You should therefore **keep a running record while developing.**

For example:

Prompt:

"Suggest a Django model structure for products, boxes and orders."

AI suggested:

Product, Box, Order, OrderItem.

Accepted:

OrderItem relationship.

Modified:

Changed Box.max_weight from FloatField to DecimalField.

Rejected:

AI suggested storing product dimensions as a JSONField.

I preferred explicit DecimalFields because dimensions are core queryable data.

That demonstrates engineering judgment.

5. Very important: the chat transcript requirement

This is probably the most dangerous part of the assignment.

The assignment says:

Export the chat transcript and attach and include it in your repository.

Then immediately says:

"Make sure that you don't generate this using AI."

And:

"Your assignment will be rejected immediately if we see that you used AI for this."

Assignment - Python_Django Deve...

Therefore:

Do not ask me to write your chat transcript.

Do not have AI:

- fabricate the transcript
- summarize the transcript and call it the transcript
- reconstruct previous conversations
- generate a fake conversation
- write a "realistic" transcript

The safest approach is to export the **actual conversation yourself** using the platform's export functionality and put that original export into the repository.

This requirement is specifically designed to detect fabricated AI-generated evidence.

6. Same issue with "What did you learn?"

The assignment says:

"What did you learn in this assignment? Again, not to use AI to generate an answer for this."

Assignment - Python_Django Deve...

So I should **not write your final** `LEARNINGS.md` **for you**.

I can help you understand:

- what concepts the assignment involves
- what you should think about
- what questions to ask yourself
- how to structure your own notes

But the final learning document should contain **your actual experience and observations**.

That distinction matters because the assignment explicitly says not to use AI to generate that answer.

7. What I think the evaluator is really testing

Although the assignment doesn't publish a scoring rubric, its wording strongly suggests several evaluation areas.

A. Django fundamentals

They want to see whether you can design a small Django system properly.

Likely areas:

- models
- relationships
- migrations
- views/API
- validation
- serializers if using DRF
- database interaction
- tests
- clean project structure

You don't need to make this an enormous application.

B. Data modeling

A sensible conceptual model could look like:



and separately:

Box

For example:

Product

name

length

width

height

weight

Box

name

internal_length

internal_width

internal_height

max_weight

cost

Order

created_at

OrderItem

order

product

quantity

This is only a conceptual example—not a requirement from the assignment.

The assignment itself doesn't prescribe your schema.

8. The hardest part: box selection logic

This is where you should spend most of your reasoning effort.

Suppose you have:

Product

10 × 8 × 5 cm

2 kg

Boxes

Small

$12 \times 10 \times 6$

max 5 kg

cost ₹20

Medium

$20 \times 15 \times 10$

max 10 kg

cost ₹35

Large

$40 \times 30 \times 20$

max 20 kg

cost ₹60

The system should determine which boxes can physically contain the product and satisfy the weight constraint.

But there is another question:

What does "fit" mean?

Dimensions have orientation.

A product:

$10 \times 20 \times 5$

could potentially fit into:

$20 \times 10 \times 5$

depending on whether rotation is allowed.

Your implementation should therefore explicitly define whether products can be rotated.

9. Don't make the classic dimension mistake

A naive implementation might do:

```
product.length <= box.length  
product.width <= box.width  
product.height <= box.height
```

That only works if the orientation is fixed.

If rotation is allowed, you need to determine whether **some permutation of the product dimensions fits the box dimensions**.

For example:

```
Product:  
10 x 20 x 5  
  
Box:  
20 x 10 x 5
```

These dimensions are compatible if rotation is permitted.

This is exactly the sort of edge case I'd expect a technical reviewer to test.

10. Weight is a separate constraint

A box can have sufficient physical dimensions but still be invalid.

For example:

```
Product:  
10 x 10 x 10  
Weight = 8 kg  
  
Box:  
20 x 20 x 20  
Maximum weight = 5 kg
```

It physically fits.

But it **cannot be selected**.

Therefore the selection logic should treat both as constraints:

```

dimension fit
    AND
weight capacity

```

Not:

```

dimension fit
    OR
weight capacity

```

11. Multiple products make the problem more interesting

The assignment talks about:

"an order"

rather than simply one product.

Assignment - Python_Django Deve...

That means you should think carefully about:

```

Order
├─ Product A × 2
├─ Product B × 1
└─ Product C × 3

```

Now you have to determine:

Total weight

Something like:

```

total_weight =
    product_A.weight * 2
    + product_B.weight * 1
    + product_C.weight * 3

```

But dimensions become more complicated.

You cannot simply do:

```

total_length = sum(lengths)
total_width = sum(widths)

```

```
total_height = sum(heights)
```

and claim that this represents actual packing.

That would be an oversimplification.

12. You need to decide the scope of packing

This is one of the biggest ambiguities in the assignment.

There are at least two possible interpretations.

Approach A — simplified bounding-box approach

Calculate an approximate combined volume/bounding dimensions.

This is relatively simple.

Approach B — actual 3D packing

Determine whether multiple objects can physically be arranged inside the box.

That is a much harder bin-packing / 3D packing problem.

The assignment does **not explicitly require a sophisticated 3D packing algorithm**.

Therefore, don't unnecessarily turn a small hiring assignment into a research project.

Instead, document your chosen assumption.

For example:

"The implementation uses a deterministic simplified packing strategy and explicitly documents its limitations rather than claiming to solve general 3D bin packing."

That is much better engineering than pretending a simplistic calculation solves arbitrary packing.

13. What "most suitable" should probably mean

Since the assignment doesn't define the exact optimization criterion, **you need to establish one**.

A sensible hierarchy could be:

1. Box must physically satisfy the requirements.
2. Box must support total weight.
3. Among valid boxes, select the smallest suitable box.
4. Use cost as a tie-breaker.

Or:

1. Valid dimensions
2. Valid weight
3. Lowest cost
4. Smallest volume as tie-breaker

But don't blindly assume either is correct.

The important thing is:

Define the rule.

Then make your tests reflect it.

For example:

Suitable boxes:

Small → valid
Medium → valid
Large → valid

If your documented policy is "smallest suitable box", then:

Small

is selected.

If your policy is "lowest cost", then cost determines the result.

The evaluator can disagree with your exact optimization policy, but they are much more likely to appreciate a **clearly defined, deterministic rule** than unexplained behavior.

14. Your algorithm should be separated from HTTP/Django code

I strongly recommend avoiding something like:

```
def recommend_box(request):  
    # 100 lines of database queries  
    # dimension calculations  
    # validation  
    # sorting  
    # HTTP response
```

Instead, think in layers:

```
HTTP/API layer  
    ↓  
Application/service logic  
    ↓  
Box selection algorithm  
    ↓  
Django ORM
```

For example conceptually:

```
recommend_box(order)
```

should be independently testable.

This makes it much easier to test the actual business logic without having to make HTTP requests every time.

15. What should NOT be over-engineered

You don't need to build:

- microservices
- Kafka
- Kubernetes
- complex authentication
- payment integration

- Redis
- Celery
- AWS deployment
- elaborate frontend
- machine learning
- an actual AI model

The assignment is fundamentally:

```
Django
+
database
+
box-selection logic
+
tests
+
documentation
```

Adding unrelated technology can actually hurt because it increases the amount of code you need to defend.

16. Don't confuse "AI-assisted" with "AI-powered"

The title is:

AI-Assisted Box Selection System

Assignment - Python_Django Deve...

But the assignment's AI section is about using AI during development.

Assignment - Python_Django Deve...

It does **not** say that your application must call:

- OpenAI
- Gemini
- Claude
- an LLM API

Therefore, don't automatically add an LLM to the application.

The "AI-assisted" aspect is primarily about **development transparency**.

17. Testing is not optional

The assignment explicitly requires:

Test cases.

and:

Test run output.

Assignment - Python_Django Deve...

Django itself provides a testing framework intended to test application behavior, including database-backed code.  Django Project +1

If you use DRF, `APITestCase` / `APIClient` can test API behavior at the HTTP/API layer.

 Django Rest Fra...

18. Your tests should test the business rules, not just endpoints

Weak testing:

```
GET /boxes/  
→ 200 OK
```

That's useful, but not enough.

You want tests such as:

Test 1 — exact fit

```
Product:  
10 × 10 × 10
```

```
Box:  
10 × 10 × 10
```

Expected:

```
valid
```

Test 2 — one dimension too large

Product:

$11 \times 10 \times 10$

Box:

$10 \times 10 \times 10$

Expected:

invalid

Test 3 — rotated fit

Product:

$10 \times 20 \times 5$

Box:

$20 \times 10 \times 5$

Expected:

valid if rotation is supported

Test 4 — overweight

Product weight = 10kg

Box capacity = 5kg

Expected:

invalid

Test 5 — multiple valid boxes

Verify your documented selection rule.

Test 6 — no suitable box

Expected behavior should be explicitly defined.

For example:

```
404 / 422 / domain-specific error
```

The exact status depends on your API design.

Test 7 — quantity

If an order contains:

```
Product A × 5
```

make sure your calculations account for all five.

Test 8 — invalid dimensions

For example:

```
length = 0  
weight = -2
```

Your system should reject invalid data.

19. Test the algorithm separately from the API

This is a particularly good engineering distinction.

You can have:

Unit tests

```
can_fit(product, box)
```

and:

```
select_best_box(order, boxes)
```

Then:

API tests

```
POST /orders/
```

```
POST /recommendation/
```

This gives you confidence at two levels:

```
Business logic
```

↓

```
HTTP/API integration
```

DRF's testing utilities are specifically designed to make API-level tests straightforward.

🔴 Django Rest Fra...

20. Your README.md should tell a reviewer how to evaluate you

A good README should probably contain:

```
# Project
```

```
## Problem
```

```
## Solution
```

```
## Assumptions
```

```
## Box Selection Strategy
```

```
## Data Model
```

```
## API Endpoints
```

```
## Setup
```

```
## Environment Variables
```

```
## Database Setup
```

```
## Running the Application
```

```
## Running Tests
```

```
## Example Request
```

```
## Example Response
```

```
## Project Structure
```

```
## Limitations
```

```
## AI Usage
```

Especially:

Assumptions

This is where you explain ambiguous things such as:

- dimension units
- whether rotation is allowed
- how multiple products are packed
- what "most suitable" means
- how ties are resolved
- what happens when no box fits

This section could become one of the strongest parts of your submission.

21. AI_USAGE.md should be an audit trail

Do not write:

```
"I used ChatGPT to help me code."
```

That's too vague.

Instead, structure it around actual events:

```
# AI Usage
```

```
## Tools Used
```

```
- ChatGPT
- ...
```

```
## Prompt 1
```

```
[actual prompt]
```

```
### Output
```

```
[what AI suggested]
```

```
### Decision
```

```
Accepted / modified / rejected
```

```
### Reason
```

```
[my reasoning]
```

```
## Prompt 2
```

```
...
```

```
## AI Mistakes
```

```
1. ...
2. ...
```

```
## Verification
```

```
- Unit tests
- Manual API testing
- Edge-case testing
- ...
```

The assignment explicitly asks for prompts, accepted/rejected output, AI mistakes, and verification.

Assignment - Python_Django Deve...

22. Don't fabricate "AI mistakes"

This is another subtle point.

If AI didn't make a mistake, don't invent one just to make `AI_USAGE.md` look authentic.

You can honestly say:

```
No critical AI-generated defect was found in this section.
```

But if you did find something, document it.

For example:

```
AI suggested comparing dimensions in their stored orientation only.
```

```
I identified that this would reject valid rotated configurations, so I modified the im
```



That is useful evidence of your own reasoning.

23. TEST_OUTPUT.md

If you don't use GitHub Actions, the assignment explicitly allows terminal output instead.

Assignment - Python_Django Deve...

For example:

```
$ python manage.py test
```

```
Found XX test(s).
```

```
Creating test database...
```

```
...
```

```
-----  
Ran XX tests in X.XXXs
```

```
OK
```

```
Destroying test database...
```

Don't manually type fake output.

Run the actual command and paste the actual result.

24. GitHub Actions is optional, but useful

The assignment gives two alternatives:

GitHub Actions link

OR

TEST_OUTPUT.md

So CI is not mandatory.

However, if you already know Jenkins/CI/CD, a small GitHub Actions workflow could demonstrate good engineering practice.

But don't add CI merely to inflate the project.

A clean:

```
.github/
├─ workflows/
│   └─ tests.yml
```

that runs:

Rash

is enough.

25. What I would consider "minimum complete"

Your project should reach this state:

```
django-box-selector/
|
├─ manage.py
├─ requirements.txt
├─ README.md
├─ AI_USAGE.md
├─ LEARNINGS.md
├─ TEST_OUTPUT.md
|
├─ chat-transcript/
│   └─ <actual exported transcript>
|
├─ .github/
│   └─ workflows/
```

```

|      └─ tests.yml      # optional
|
|─ config/
|
|─ boxes/
|   └─ models.py
|   └─ serializers.py    # if DRF
|   └─ views.py
|   └─ urls.py
|   └─ services.py
|   └─ tests/
|       └─ test_models.py
|       └─ test_selection.py
|       └─ test_api.py
|   ...

```

The exact structure can differ.

The important thing is **separation of concerns and understandable code**.

26. Things I would specifically avoid

✗ Fake AI transcript

The assignment explicitly prohibits AI-generated transcript evidence.

Assignment - Python_Django Deve...

✗ AI-written LEARNINGS.md

The assignment explicitly says not to use AI for this.

Assignment - Python_Django Deve...

✗ Fake test output

Run the tests yourself.

✗ Undefined selection logic

Don't just write:

Python

Run

and call it "best box."

✗ Hardcoded box IDs

Avoid:

```
if order.total_weight < 5:  
    return Box.objects.get(id=1)
```

✗ Fixed dimension orientation without explanation

You need to consciously decide how rotation works.

✗ Pretending to solve general 3D packing

If your algorithm is simplified, document its scope.

✗ Excessive architecture

Don't build a distributed system for a small Django assignment.

✗ Unnecessary AI API integration

Nothing in the assignment requires an LLM inside the product.

✗ No tests for edge cases

This is probably one of the easiest ways to make a technically functioning solution look weak.

27. The assignment has several ambiguities you should explicitly resolve

This is perhaps the most valuable part of the exercise.

Question	You need a decision
Can products rotate?	Yes/no
What are dimension units?	cm/mm/etc.
Can multiple products share one box?	Define behavior
How is combined volume/packing handled?	Define algorithm

Question**You need a decision**

What is "most suitable"?

Define optimization

What happens if nothing fits?

Define response

Can a product exactly equal box dimensions?

Usually yes

Can weight equal maximum capacity?

Usually yes

Are zero/negative dimensions valid?

No

Are fractional dimensions allowed?

Define

Are fractional weights allowed?

Define

How are ties resolved?

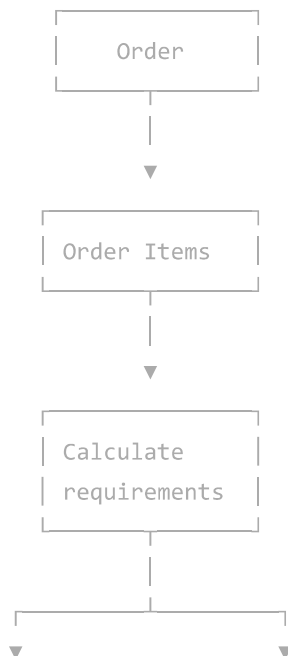
Define deterministic rule

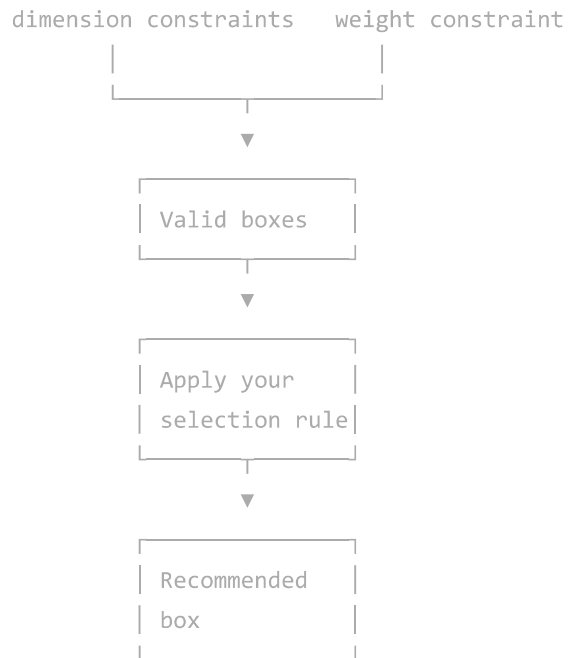
None of these should be hidden inside the code.

Put them in `README.md` under **Assumptions / Design Decisions**.

28. A good mental model for the entire assignment

Think of the system as:





That is the core of the assignment.

Everything else—Django, API, database, tests, README, CI—is supporting that core.

29. What I would prioritize if this were your submission

Given that you already know Django/Python/REST concepts, I would **not** spend most of your time learning Django again.

I'd focus on:

Priority 1 — Define the algorithm

Write down, in your own words:

What constitutes a valid box?

How do dimensions work?

How does rotation work?

How are multiple products handled?

What makes one valid box better than another?

Priority 2 — Build the domain model

Keep it simple and normalized.

Priority 3 — Implement the selection logic independently

Make it deterministic and testable.

Priority 4 — Build the Django/API layer

Expose the functionality cleanly.

Priority 5 — Test edge cases

Especially dimension orientation, weight, quantity, exact fits, and no-fit situations.

Priority 6 — Documentation

Explain your decisions.

Priority 7 — AI evidence

Document what **actually happened** during development.

Priority 8 — Final verification

Fresh clone → install → migrate → test → run.

That last step is important because it proves the repository works independently of your development machine.

30. One thing I would change from a normal project workflow

For a normal project, you might use AI freely and clean up everything afterward.

For **this assignment**, don't do that.

Keep an audit trail from the beginning:

```
Your thought
↓
AI suggestion
↓
Your evaluation
↓
Your modification
```

↓
Your test
↓
Final implementation

That is almost certainly what the employer means by:

"your submission must clearly show your own thinking."

Assignment - Python_Django Deve...

Bottom line

The assignment is relatively small technically, but **strict about authenticity and engineering reasoning**.

The actual deliverable is effectively:

A working Django box-selection system + a defensible selection algorithm + meaningful tests + documentation of your decisions + honest evidence of how AI was used.

The two things you should **not outsource to me** are the **actual chat transcript** and your **personal** `LEARNINGS.md` **answer**, because the assignment explicitly prohibits AI generation for those.

Assignment - Python_Django Deve...

For everything else, I can help you reason through the design, identify edge cases, review your implementation, and verify whether the solution actually satisfies the assignment.

Sources



gave explanations, code and examples. I filtered them and kept

only the information I needed.

Yes. If you're describing your AI usage in `AI_USAGE.md`, that sentence is reasonable. A slightly clearer version would be:

The AI provided explanations, code suggestions, and examples. I reviewed the responses, filtered out unnecessary information, and kept only the parts relevant to my assignment.

For `AI_USAGE.md`, I'd make it more explicit about your own judgment:

The AI provided explanations, code suggestions, and examples. I reviewed and filtered the responses, kept only the information relevant to the assignment, and modified or rejected suggestions when they did not fit my requirements.

The second version better demonstrates the “**your own thinking**” requirement in the assignment.

Assignment - Python_Django Deve...

Sources



ChatGPT can make mistakes. Check important info.